RAZ Systems, LLC

We Do AI.

Agentic AI Enginnering Capstone Project

Project Requirements — 401(k) & Compensation Assistant

Raz Systems · Agentic AI Engineering Capstone Project

Please complete 4_LangGraph_401k_RAG_Email_Project_skeleton.ipynb

To be Submitted by Jun 19 2026 9:00 AM. Email the project to training@razsystems.com

Email Subject : Capstone, [YOUR FULL NAME]

Below are the skills covered in this project:

LangGraph state management — defining a TypedDict state schema with multiple fields beyond just messages (wants_email, subject, html_body, approved)

Tool-calling with ToolNode — binding multiple heterogeneous tools to an LLM and routing with tools_condition-style logic

Real vector-database RAG — Supabase/pgvector integration: embedding a query, calling a Postgres RPC function, and ranking/formatting results by similarity

Mocking a third-party API — simulating an external vendor (Prudential) with keyword matching and a fallback strategy, in the absence of a live API

Identity-scoped data access — designing a tool (get_my_compensation) that takes no free-form arguments and always resolves to the logged-in user, preventing prompt-injection-style access to other users' data

SQLite integration — CRUD operations keyed by a natural identifier (email) rather than a synthetic account number

Multi-agent pipelines vs. tool-calling — recognizing when a task is a fixed-order sequence (subject line → HTML formatting → send) versus a genuine tool-selection decision

Human-in-the-loop patterns — a synchronous input()-based approval gate as the simplest form of HITL, without checkpointer/interrupt() machinery

Three-way conditional routing — extending the standard two-way (tool vs. end) router into a three-way branch (tool / email pipeline / end), with correct ordering of checks

Real external service integration — SendGrid email sending via the actual SDK shape, with graceful fallback/simulation when no API key is present

Graph composition — assembling a full StateGraph from previously-independent building blocks (RAG, tool-calling, multi-step pipeline, HITL) into one coherent agent

(Bonus) Observability — LangSmith tracing setup, and reflecting on the difference between logging and tracing

1. Project Overview

Your company's HR department wants a chat-based assistant that employees can use to:

- Ask about **company HR policy** (benefits, leave, 401k rules) — answered from the official handbook
- Ask about **their own pay and 401(k) contribution —** but only ever *their own*, never anyone else's
- Ask questions about the **401(k) plan itself** as administered by the third-party vendor (Prudential) — matching formulas, vesting, withdrawal rules, contribution limits
- **Request an email summary** of any of the above, sent only after a human reviews and approves it

You are building this assistant as a **LangGraph agent**. This document specifies what the system must do. It does not tell you how to write the code — that's the assignment. Use this document to understand the requirements *before* you open the notebook, and refer back to it if you get stuck on what a piece is supposed to accomplish (as opposed to how to implement it).

2. Functional Requirements

FR1 — Authentication

The system must identify *who* is asking before answering any compensation-specific question. A user must log in with an email and password before the assistant will run. The logged-in email is the only identity used for the rest of the session — it is never asked for again, and it is never supplied by the LLM.

FR2 — Policy Q&A (RAG)

The assistant must be able to answer questions about HR/401(k) policy by retrieving relevant content from a **vector database**, not by the LLM guessing from its own training data. If no relevant content is found, the assistant must say so rather than fabricating an answer.

FR3 — Personal Compensation Lookup

The assistant must be able to report the **logged-in user's own** salary and 401(k) contribution rate. It must be structurally impossible for this feature to return another employee's data — not just "the LLM is told not to," but designed so there is no parameter through which another person's identity could even be supplied.

FR4 — Third-Party Vendor Information

The assistant must be able to answer questions about how the 401(k) plan itself works — matching formula, vesting schedule, early withdrawal rules, annual contribution limits — sourced from the plan's third-party administrator (Prudential), as distinct from the company's own internal HR policy (FR2).

FR5 — Tool Selection

The assistant must automatically choose which of FR2/FR3/FR4 applies to a given question, without the user specifying which "mode" they want. A single natural-language question should route itself correctly.

FR6 — Email Summary (Explicit Trigger Only)

The assistant must be able to take any answer it has already produced and deliver it as an email — but **only when the user explicitly asks for this**. A plain question must never trigger an email. The system must not guess that the user "probably wants this emailed."

FR7 — Email Composition

The email must have a **subject line** appropriate to its content, and a **body formatted as HTML** suitable for an email client — not a copy-paste of the raw chat answer.

FR8 — Human Approval Before Sending

No email may be sent without a human explicitly approving it first. The system must show the human exactly what will be sent (recipient, subject, body) before asking for approval, and must not send anything if approval is withheld.

FR9 — Actual Delivery

Once approved, the system must attempt to send the email through a real email delivery provider (SendGrid). If no provider credentials are configured, the system must clearly indicate that sending was simulated, rather than silently doing nothing or crashing.

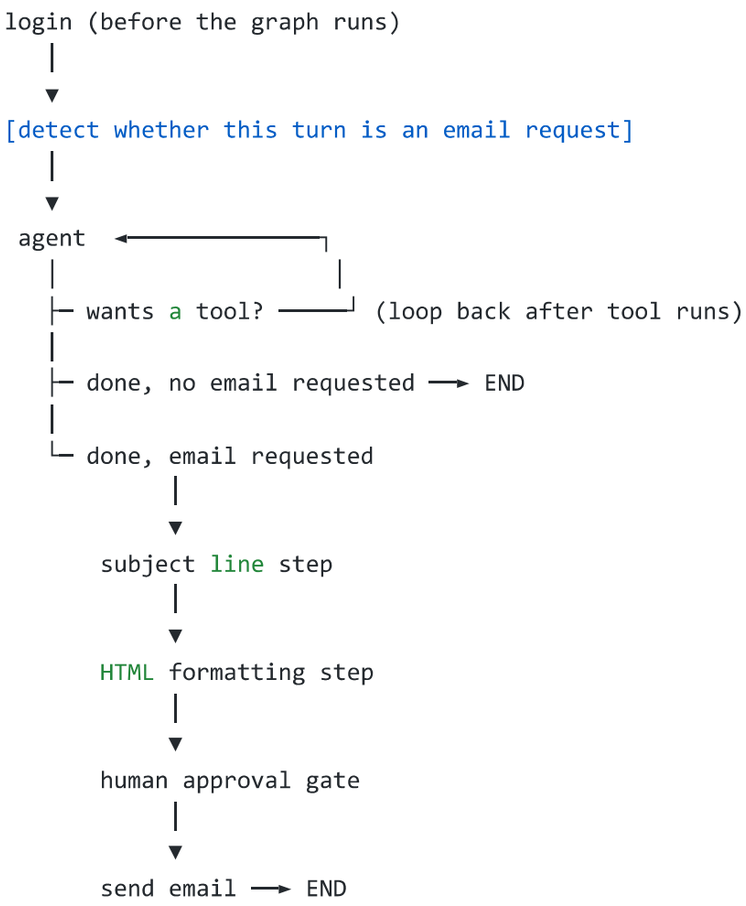
3. Non-Functional / Architectural Requirements

- **NFR1 — Single graph.** All of the above must be expressed as one LangGraph `StateGraph`, not as separate scripts or separate `if` branches outside the graph.
- **NFR2 — Reuse the ToolNode pattern.** FR2, FR3, and FR4 must be implemented as tools bound to one LLM and executed through **one** `ToolNode`, regardless of how different their internal implementations are (a database query, a vector search, and a static lookup are very different internally — they must still share one tool-execution node).
- **NFR3 — The email sequence is not tool-calling.** FR6–FR9 must not be modeled as something the LLM "decides to call" the way FR2–FR4 are. They are a fixed sequence: once triggered, every step always runs in the same order.
- **NFR4 — Hard correctness guarantees stay in code.** Anything that must be guaranteed (e.g. "never send without approval," "never look up someone else's salary") must be enforced by Python control flow or function signatures — never left as an instruction inside a prompt that the LLM is merely *asked* to follow.

4. Architecture You're Expected to Land On

Without giving away the implementation, here is the shape of the solution:

code



Two distinct "decision mechanisms" appear in this graph, and recognizing which is which is the central lesson of this project:

Mechanism	Used for	What decides
Tool-calling (`bind_tools` + `ToolNode`)	FR2, FR3, FR4	The LLM, per question, chooses zero or one tool
Fixed sequence (plain edges)	FR6–FR9	Nothing decides — every step always runs once triggered

If you find yourself trying to make the email steps into tools the agent "calls," stop — that's a sign you've misread which mechanism this requirement needs. Re-read NFR3.

5. Hints by Component

Hint set A — Login (FR1)

- This does not need to be a graph node. The graph doesn't run until *after* someone is logged in, so a plain Python function called before `app.invoke(...)` is the right shape.
- Store the result (the verified email) somewhere the rest of your code can reach it without it being passed around as a tool argument — a module-level variable is the simplest option for this scale of project.

Hint set B — Policy RAG tool (FR2)

- You've already built this exact retrieval pattern before (HR Handbook RAG system). The only thing new here is which table/RPC you're calling, not the technique.
- Decide what your tool should return if the vector database returns zero matches — re-read FR2's second sentence. What does "rather than fabricating an answer" imply your tool's return value should look like in that case?
- Your tool's docstring is what the LLM reads to decide *when* to call this tool. Make sure it clearly distinguishes "company policy" language from "the plan vendor" language — otherwise you risk the LLM confusing this tool with the one for Hint Set D.

Hint set C — Personal compensation tool (FR3)

- Re-read FR3 carefully: "structurally impossible," not "instructed not to." What does a tool's function *signature* look like if it's designed so there is no parameter for "whose data to look up"?
- If your function has zero parameters, how does it know which row to query? Think back to Hint Set A — where did you store the logged-in email?

Hint set D — Vendor info tool (FR4)

- This is a mocked tool, not a real API call — same idea as the original HR policy mock you built earlier in this curriculum, just representing a different organization (Prudential) instead of your own HR department.
- A simple keyword-to-response lookup is enough here. The interesting design question isn't the matching logic — it's making sure this tool's docstring makes clear to the LLM that it's a *different* source of information than Hint Set B's tool (internal policy vs. third-party plan administrator), even though both answer "401k" questions.

Hint set E — Tool selection (FR5)

- This requirement isn't something you implement directly — it falls out of B, C, and D being correctly written as `@tool`-decorated functions with clear, distinguishing docstrings, all bound to the same LLM via `bind_tools`.
- If FR5 doesn't seem to work correctly when you test it (wrong tool gets picked), the bug is almost always in the docstrings being too similar or too vague, not in any routing logic you wrote yourself — there isn't any routing logic you write yourself for this part.

Hint set F — Detecting an email request (FR6)

- This has to run *before* you know what the final answer even is — you're detecting something about the user's *question*, not about the answer.
- A small, separate node that runs once per turn and sets a flag in state is enough. It doesn't need an LLM call; simple keyword matching is a legitimate solution at this scale (though you may reasonably argue for an LLM-based classifier instead — both are defensible; the requirements don't mandate one approach).
- Where does this flag need to live so that a *router function* several steps later can still see it? Think about what `state` actually is.

Hint set G — The three-way router

- Every earlier project in this curriculum had a router with two destinations. This one needs three. Think about what question, in order, the router needs to ask:
  1. Did the agent just request a tool? (if yes, that takes priority over everything else)
  2. If not — is this turn an email request?
  3. If neither — just end.
- Get the order of these checks wrong and you'll see one of two bugs: either the email pipeline fires too early (mid-tool-call), or it never fires at all.

Hint set H — Subject line and HTML formatting (FR7)

- Both of these are just one more `llm.invoke(...)` call each, with a prompt — there's no new LangGraph concept here, just two more nodes in a row.
- What does each of these nodes need to read from `state` to do its job? The subject-line step needs the *answer*. The HTML step needs the answer *and* (arguably) the subject. Decide what belongs in each prompt.

Hint set I — Human approval (FR8)

- The simplest version of this requirement is a node that prints what's about to be sent and waits for console input — nothing about LangGraph's more advanced interrupt/checkpoint machinery is required to satisfy FR8 as written.
- Whatever this node decides, it needs to be readable by the node *after* it (the send step) and by anyone inspecting the final state. Where does that decision need to be stored?

Hint set J — Sending the email (FR9)

- Re-read FR9's second sentence closely: there are three possible situations your send node needs to handle correctly — approved-and-configured, approved-but-not-configured, and not-approved. Make sure your node's logic actually distinguishes all three; it's easy to accidentally only handle two of them.
- If you get a `403 Forbidden` error from the real SendGrid API while testing with a real key, that's not a bug in your LangGraph code — it means your sender email isn't verified in your SendGrid account, or your API key lacks Mail Send permission. Check your SendGrid dashboard's Sender Authentication settings before assuming your Python is wrong.

6. Definition of Done

Your solution satisfies this project's requirements when:

- [ ] Logging in with valid credentials succeeds; invalid credentials are rejected (FR1)
- [ ] A policy question is answered using retrieved content, with a clear "not found" fallback when nothing relevant exists (FR2)
- [ ] A compensation question returns only the logged-in user's own data, via a tool with no identity-bearing parameter (FR3)
- [ ] A 401(k) plan-mechanics question is answered from the mocked vendor tool, distinguishably from FR2's answers (FR4)
- [ ] Three different questions, asked in any order, each correctly trigger the right one of the three tools without you telling the system which one to use (FR5)
- [ ] A plain question never produces an email; only a message that explicitly asks for one does (FR6)

- [ ] Every triggered email has a real subject line and an HTML body, not raw chat text (FR7)
- [ ] The system shows the full draft and explicitly asks for approval before any send attempt; declining prevents sending (FR8)
- [ ] An approved email either sends for real or clearly states it was simulated, depending on whether credentials are configured (FR9)
- [ ] All of the above happens inside one compiled `StateGraph` (NFR1–NFR4)
